Simulation Modeling

Systems Modeling and Simulation

4DV650

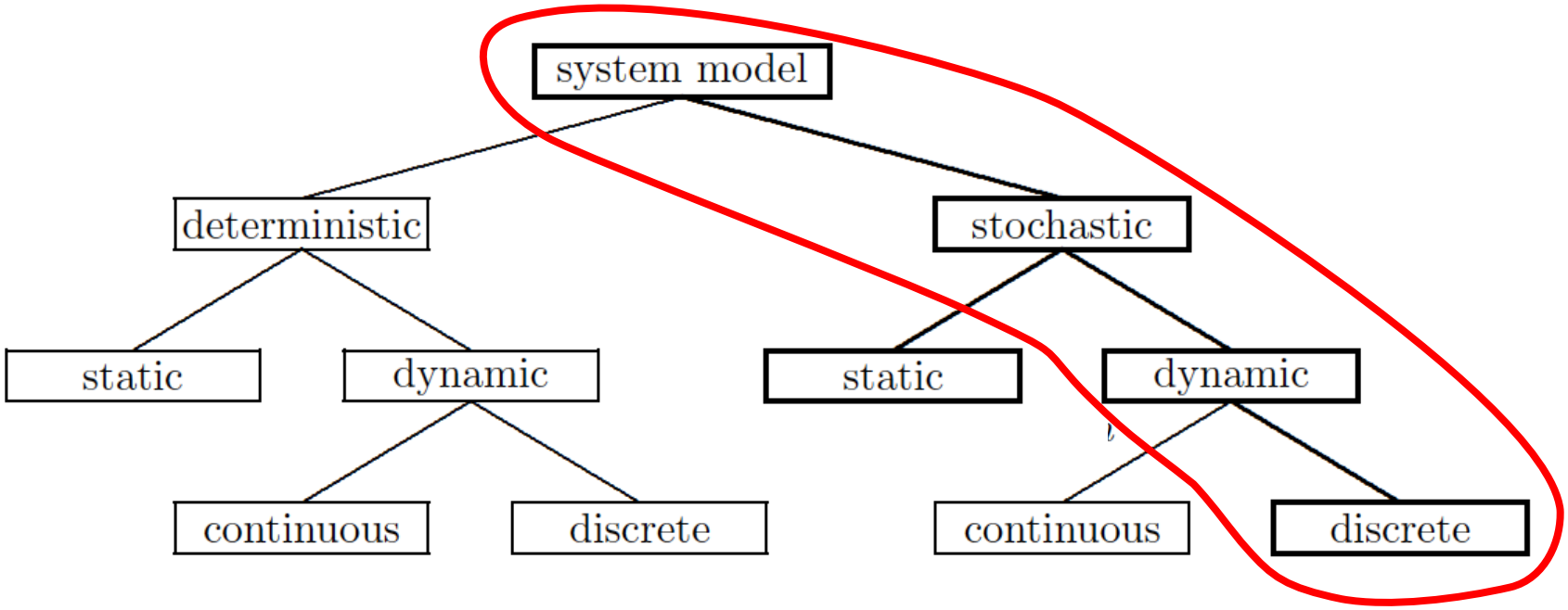
Mauro Caporuscio

Department of Computer Science

mauro.caporuscio@lnu.se

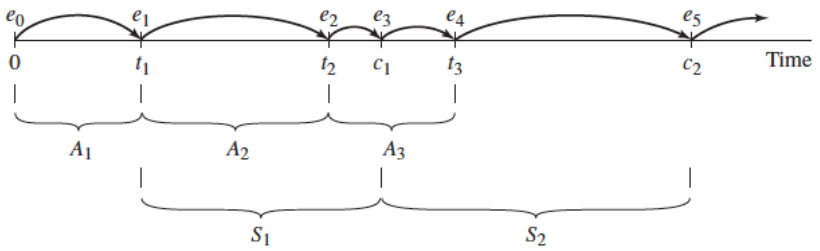


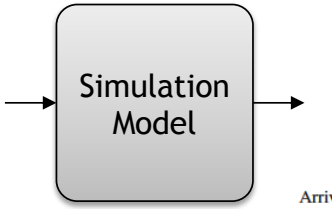
What we focus on

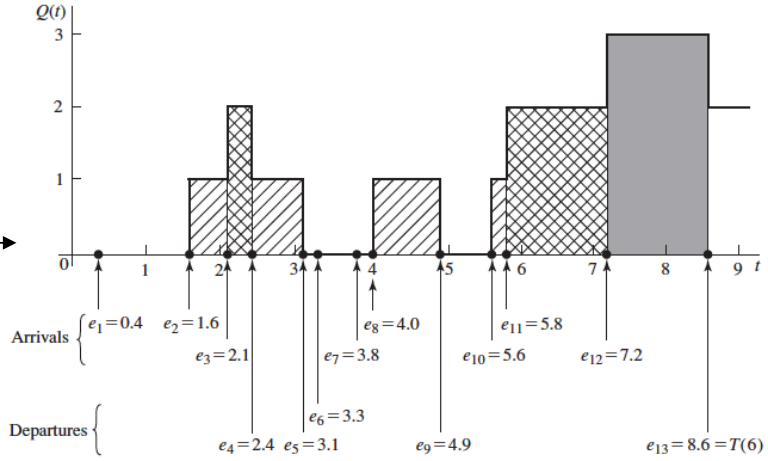


Discrete-event simulation

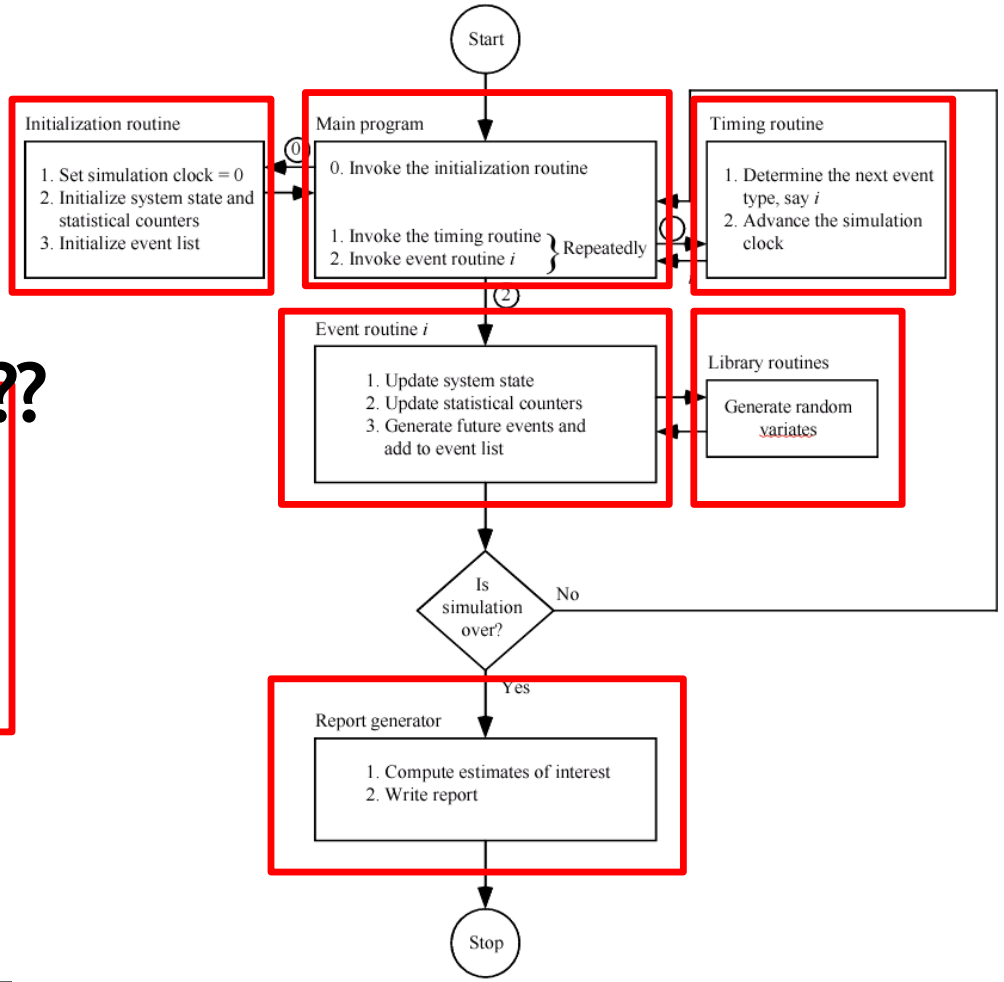
- **Idea:** The system changes instantaneously in response to certain **discrete events**
- **Definition:** A *discrete-event simulation model* is defined by three attributes:
  - ***Stochastic*** – at least some of the **system state variables** are random
  - ***Dynamic*** – the **time evolution** of the system state variables is important
  - ***Discrete-event*** – significant changes in the system state variables are associated with **events** that occur at **discrete time** instances only







General Organization of a Discrete-Event Simulation Model



??

Clock: current value of simulated time

System state: variables to describe state
Event list: times of future events

Statistical counters: accumulate quantities for output

Time-Advance Mechanisms

Time-Advance Mechanisms - Next-event time advance

- **Simulation clock**: keeps the current value of (simulated) time in the model
  - Must decide on, be consistent about, **time units**
  - Usually no relation between simulated time and (real) time needed to run a model on a computer
- **Next-event time advance** (usually used in simulation)
  1. Initialize simulation **clock** to 0
  2. Determine times of occurrence of future events – ***event list***
  3. Clock advances to **next event** (most imminent), which is executed
     - Event execution may involve updating event list
  4. Continue until **stopping rule** is satisfied (must be explicitly stated)

NB: Clock **jumps** from one event time to the next

- periods of inactivity are ignored

Time-Advance Mechanisms - Example: Single-server Queue

e_i = time of occurrence of the ith event (of any type), i = 1, 2, 3, …

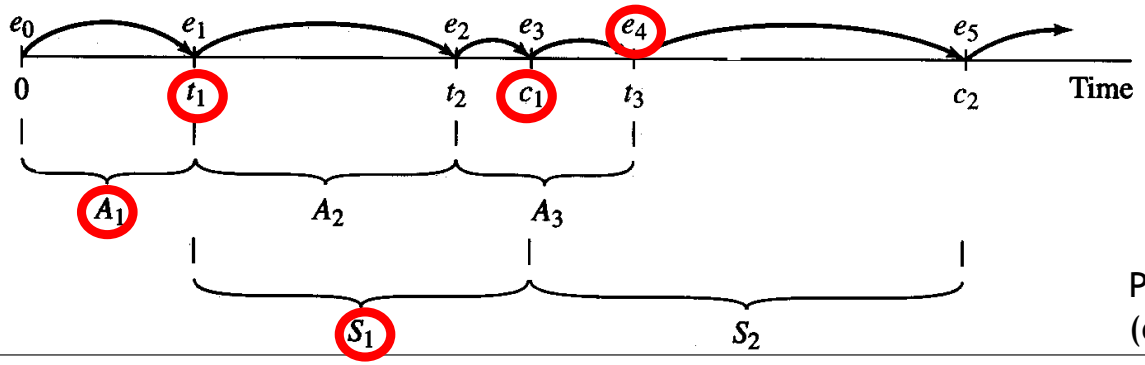
t_i = time of arrival of ith customer (t_0 = 0)

$A_i = t_i - t_{i-1}$ = interarrival time between (i-1)th and ith customers (usually assumed to be a random variable from some probability distribution)

S_i = service-time requirement of ith customer (another random variable)

D_i = delay in queue of ith customer

$c_i = t_i + D_i + S_i$ = time ith customer completes service and departs



Possible trace of events
(detailed narrative in reading)

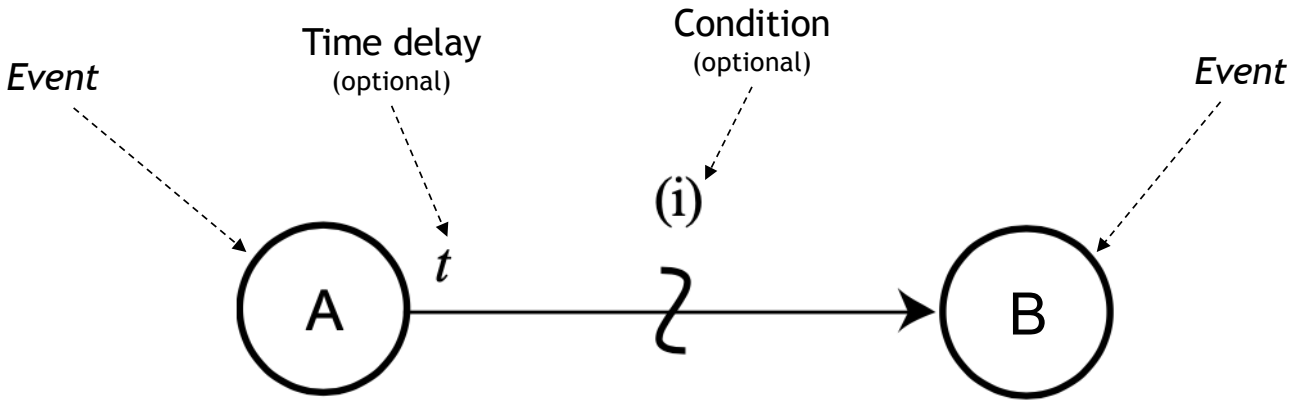
Event Graph Modeling (basics)

Determining the Events and Variables

- When dealing with complex systems the following questions arises:
  - *How to determines the **number** and **definition** of events in general for a model?*
  - *How to specify the **state variables** needed to keep the simulation running in the correct event sequence and to obtain the desired output measures?*
- Unfortunately, there is **no general way** to answer these questions, and different ways of representing *events* and *variables* may be correct
- **Event-graph** method gives formal graph-theoretic method of analyzing event structure

Event-Graph

- **Event graphs** connect the events by arcs indicating the type of event scheduling that can occur
  - **Nodes** identify Events
  - **Direct arcs** (arrows) specify how events may be scheduled from other events

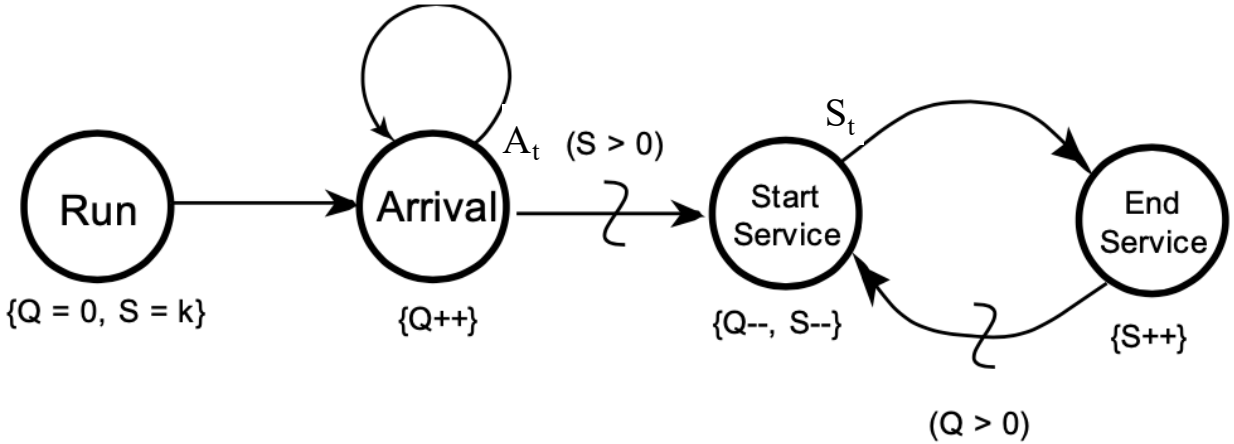


- Reads: *Event A* causes *Event B* to be scheduled after a delay of *t*, if condition *(i)* is true

Event-Graph - Example

- **Description**: Customers arrive to a service facility and are served by one of *k* servers. If all servers are busy, costumers wait in a single queue.

- **Parameters:**
  - $\{A_t\}$: interarrival times
  - $\{S_t\}$: service times
  - $k = 1$: total number of servers
- **State Variables**
  - Q: # of customers in queue
  - S: # of available servers

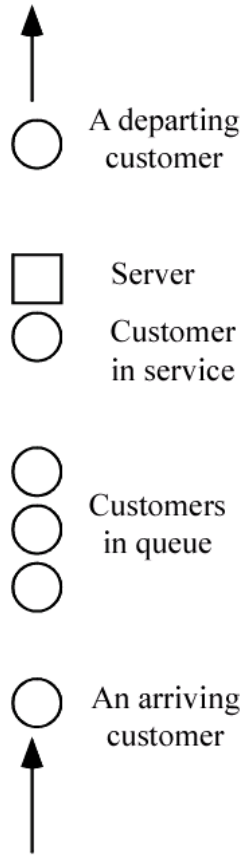


NB: The Event Graph captures the *scheduling dependencies* of the events in the model, not the flow of customers or entities through the system

Statistics

Discrete-Event Simulation - Single-server queue

- **Time Evolution**
  - ***Time of arrival***, ***Delay in queue, Service Completion,*** for each customer
- **State variables**
  - ***Status of server*** (idle, busy) – needed to decide what to do with an arrival
  - ***Length of the queue*** – to store an arrival that must wait in line
- **Events**
  - ***Arrival*** of a new customer
  - ***Departure*** of a customer
- **Objective:**
  - *Estimate expected average delay in queue*
  - *Estimate expected average number of customers in queue*
  - *Estimate expected server utilization*



Expected average delay in queue (excluding service time)

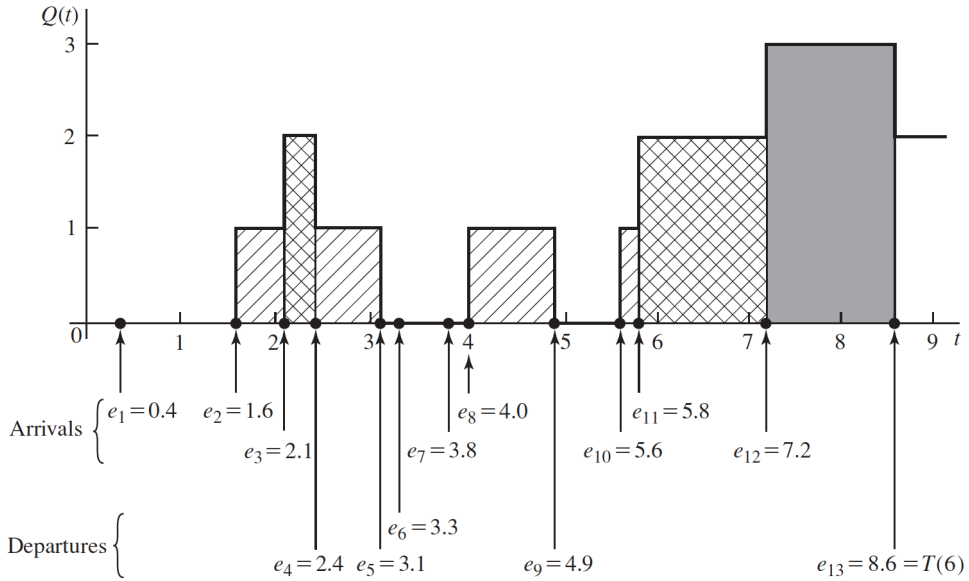
- A *Discrete-time statistic:* defined on a collection of discrete-time random variables
- Given a a run of the simulation: Let D_1, D_2, ... , D_n, be the resulting customer delays. So, the expected average delay in queue is

$$\hat{d}(n) = \frac{\sum_{i=1}^{n} D_i}{n}$$

Calculated as the average of the *n D*'s observed in the simulation

Expected average number of customers in queue

- A *continuous-time statistic*: defined on a collection of continuous-time random variables

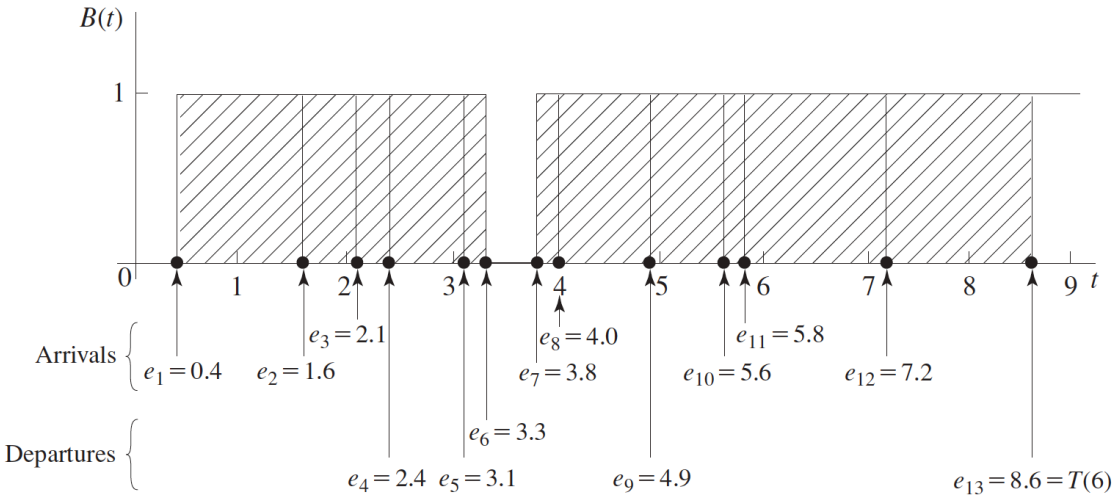


Area under $Q(t)$ = queue length at time t divided by $T(n)$ = time simulation ends

$$\sum_{i=0}^{\infty} iT_i = \int_0^{T(n)} Q(t)\,dt \qquad \hat{q}(n) = \frac{\int_0^{T(n)} Q(t)\,dt}{T(n)}$$

Expected utilization (proportion of time busy) of the server

- Another *continuous-time statistic*



Area under $B(t)$ = server-busy function (1 if busy, 0 if idle at time t), divided by $T(n)$

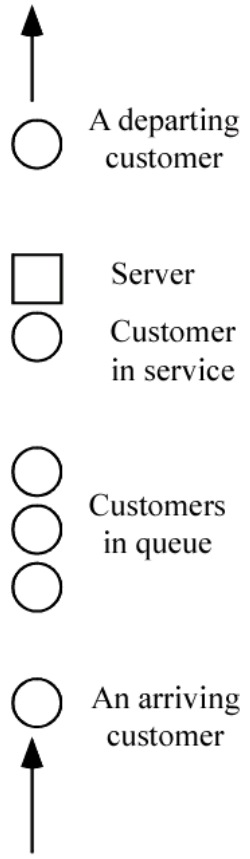
$$\hat{u}(n) = \frac{\int_0^{T(n)} B(t)\, dt}{T(n)}$$

Though simple, it contains many features found in all simulation models

Simulation of a Single-Server Queuing System

Problem Statement

- Single-server queueing model (M/M/1)
- Assume **interarrival times** are **independent and identically distributed** (IID) random variables
- Assume **service times** are **IID**, and are independent of interarrival times
- Queue discipline is **FIFO**
- Start **empty** and **idle** at time 0
- First customer arrives **after an interarrival time**, not at time 0
- **Stopping rule**: When *n*th customer has completed delay in queue (i.e., *enters* service) … *n* will be specified as input



Program Organization and Logic

- Java program to do this model (C program is in book)
  - Event types: '0' for arrival, '1' for departure
  - Modularize for main, initialization, timing, events, library, report
  - Stopping rule: ***n* = 1000** costumers
  - **Interarrival** and **service times** "drawn" from an **exponential distribution**
    - mean β = 1 for interarrivals, 0.5 for service times
    - Density function

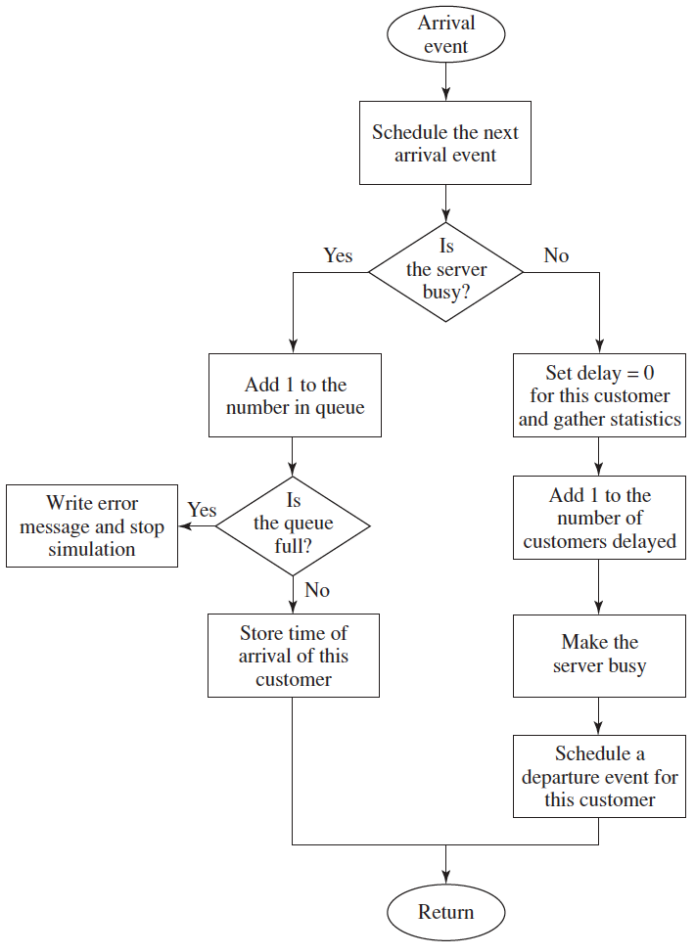
$$f(x) = \begin{cases} \frac{1}{\beta} e^{-x/\beta} & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

    - Cumulative distribution function

$$F(x) \equiv P(X \leq x) = \int_{-\infty}^{x} f(t)\,dt = \begin{cases} 1 - e^{-x/\beta} & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$


it's DEMO time!
VHF
UHF
STAND BY
VOLUME
CONTRAST
BRIGHT

Arrival routine - queueing model




```
private void arrive() {

    // Schedule next arrival.
    this.event_list[0] = this.clock + ExponentialDistribution.random(this.mean_interarrival);


    // Check to see whether server is busy
    if (server_status == BUSY) {

        // Server is busy, so increment number of customers in queue
        this.num_in_q++;

        // Check to see whether an overflow condition exists.
        if (num_in_q > Q_LIMIT) {
            // The queue has overflowed, so stop the simulation.

            this.log("\nOverflow of the array time_arrival at time " + clock);

            System.exit(2);
        }

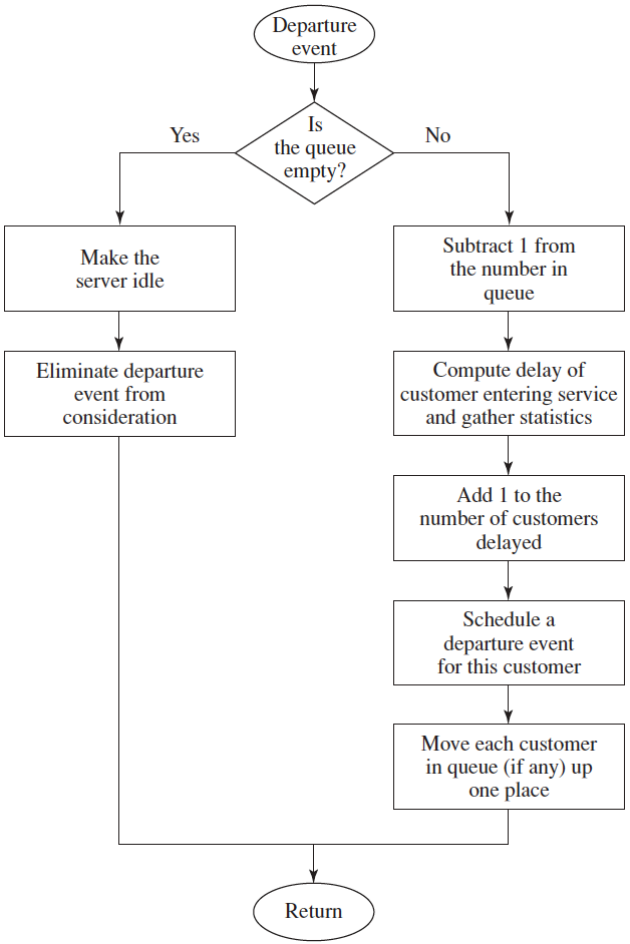
        //There is still room in the queue, so store the time of arrival of the arriving customer at the (new) end of time_arrival.
        this.time_arrival[num_in_q] = this.clock;

    }else {

        // Increment the number of customers delayed, and make server busy.
        this.num_delayed++;
        this.server_status = BUSY;

        // Schedule a departure (service completion).
        this.event_list[1] = this.clock + ExponentialDistribution.random(mean_service);
    }
}
```

Departure routine - queueing model




```java
private void depart() {

    //Check to see whether the queue is empty.
    if (this.num_in_q == 0) {
        //The queue is empty so make the server idle and eliminate the departure (service completion) event from consideration.
        this.server_status = IDLE;
        this.event_list[1] = Double.MAX_VALUE;

    }else {
        //The queue is nonempty, so decrement the number of customers in queue.
        this.num_in_q--;

        // Compute the delay of the customer who is beginning service and update the total delay accumulator.
        double delay = this.clock - this.time_arrival[0];
        this.total_delay += delay;

        //Increment the number of customers delayed, and schedule departure.
        this.num_delayed++;

        //Schedule a departure event for this customer
        this.event_list[1] = this.clock + ExponentialDistribution.random(mean_service);

        // Move each customer in queue (if any) up one place.
        for (int i = 0; i < this.num_in_q; i++)
            this.time_arrival[i] = this.time_arrival[i + 1];

    }
}
```

Random Variate Generator

```java
package generators.random.numbers;

import java.util.Random;

public class SimpleRandomGenerator {

    private Random r;

    public SimpleRandomGenerator(long seed) {
        this.r = new Random(seed);
    }

    public double random() {
        return r.nextDouble();
    }
}
```

```java
package generators.random.variate;

import generators.random.numbers.SimpleRandomGenerator;

public class ExponentialDistribution {

    SimpleRandomGenerator rg;

    public ExponentialDistribution(long seed) {
        rg = new SimpleRandomGenerator(seed);
    }

    // Exponential variate generation function
    public double random(double mean) {

        /* Return an exponential random variate with mean "mean". */
        return -mean * Math.log(rg.random());
    }

}
```

Materials (MyMoodle)

- Reading
  - 05.1 [Lecture] Basic Simulation.pdf
  - 05.2 [Lecture] Basic Event Graph Modeling.pdf
  - 05.3 [Lecture] Complex Systems.pdf
- Exercises (with solutions):
  - 05.1 [Lecture] Basic Simulation - Exercises.pdf
  - 05.1 [Lecture] Basic Simulation - Solutions.pdf
- Programming
  - 05.4 [Lecture] Systems Simulation.zip
  - SIMLIB (See 05.3 [Lecture] Complex Systems.pdf)